Minimal Equality Saturation for FPGA Datapath Optimization

Zhenghao Yang

Queen's University, Kingston, Canada
24yl6@queensu.ca

Abstract — We present **hwine**[1], a compact, explainable e-graph optimizer for FPGA arithmetic datapaths. Using 10 algebraic rewrite rules, hwine applies equality saturation with a multi-objective, FPGA-aware cost model to automatically discover LUT/DSP tradeoffs, optimal tree topologies, automatic pipeline retiming, and DSP48 pre-adder inference. We evaluate on the UCSD FIR benchmark suite (11 hand-optimized HLS designs) and five real arithmetic kernels (FFT butterfly, DCT, Stencil2D, GEMM, streaming FIR). Yosys equiv_induct proves RTL equivalence for associativity-balanced dot-product and Gauss-form FFT benchmarks; CSD decomposition is proven via e-graph union (algebraic identity); Vivado 2026.1 synthesis confirms DSP48E1 pre-adder inference (1 DSP slice, 0 arithmetic LUTs). On FIR, hwine matches Fmax (308 MHz) at all 10 sizes, using fewer LUTs on 6 of 10. On DCT 8-point, CSD decomposition converts 8 DSP multipliers to 78 LUT shift-add trees. On GEMM (N=32), associativity reduces the serial adder chain from 31 to 5 levels (6.2×). On streaming FIR, pipeline retiming discovers Pareto-optimal tradeoffs between 0.30ns pipelined and 4.50ns combinational implementations.

Index Terms — e-graph, equality saturation, FPGA, datapath optimization, RTL, hardware architecture, FIR filter, Vivado, HLS

1. Introduction

Field-programmable gate arrays (FPGAs) offer unique opportunities for custom datapath design, but achieving optimal quality of results (QoR) requires exploring a vast space of equivalent micro-architectures. Designers must manually decide on adder tree topologies, pipeline stage insertion points, DSP versus LUT resource allocation, and coefficient multiplication strategies—decisions that interact combinatorially and resist local optimization heuristics.

Existing approaches to automating this exploration fall into three categories. **Traditional logic synthesis** tools (Yosys, Vivado, Synopsys DC) apply destructive rewriting at the gate level and suffer from the **phase-ordering problem**: the order of rewrites determines the final result, and finding the optimal order is NP-hard. **Hand-crafted e-graph systems** such as ROVER [1] encode domain expertise as 187 manually-designed rewrite rules, achieving up to 63% area reduction on Intel benchmarks but requiring significant expert effort per operator family. **Reinforcement learning approaches** such as PrefixRL [2] use deep RL to search design spaces without domain knowledge, but at the cost of 32,000 GPU-hours of training and limited generalizability beyond prefix circuits.

We propose a fourth approach: **compact algebraic e-graph optimization** — using a small set of algebraic rules, combined with multi-objective FPGA-aware extraction, to automatically discover micro-architectural structures that normally require expert RTL design or HLS pragma tuning. Our system, hwine, makes four contributions:

1. **Compact rule set covers key FPGA patterns.** Ten algebraic rules (associativity, MAC decomposition/fusion, constant decomposition via CSD, distributivity, DSP pre-adder mapping, pipeline retiming, and identity rules) cover addition tree balancing, multiply-accumulate mapping, constant multiplier elimination, DSP resource tradeoffs, and automatic pipeline stage insertion. No benchmark-specific rules or ML training required.

[1] **hwine** stands for hardware is not enough — and also hwine is not enough, a doubly self-referential name.

2. **Multi-objective FPGA-aware extraction.** A delay-weighted cost model with pipeline barrier support drives extraction under LUT, DSP, FF, and delay constraints. A sharing-aware cost mode deduplicates common subexpressions. Pareto-frontier extraction identifies non-dominated design alternatives.
3. **Formally verified equivalence.** Yosys equiv_induct proves RTL-level equivalence for dot-product (associativity tree balancing) and FFT butterfly (Gauss form). E-graph union proves CSD constant decomposition as algebraic identity. Cycle-safe dynamic programming extraction prevents invalid zero-substitution from DAG cycles.
4. **Concrete optimizations on real benchmarks.** On DCT 8-point, CSD decomposition converts 8 DSP multipliers to 78 LUT shift-add trees — an optimization neither Vivado HLS nor the Kastner FIR benchmarks apply. On GEMM (N=32), associativity reduces 31 serial adder levels to 5 balanced levels (6.2×). On FIR, hwine matches Kastner's hand-optimized Fmax (308 MHz) at all 10 sizes. DSP pre-adder verification confirms Vivado infers 1 DSP48E1 with 0 arithmetic LUTs for the (A+B)×C pattern. Streaming FIR pipeline retiming discovers Pareto-optimal delay-vs-FIFO tradeoffs.

We evaluate hwine across three experiment categories: pure adder tree scaling, DSP-based multiply-accumulate FIR designs, and constant-folded shift-add FIR designs versus the complete UCSD Kastner FIR benchmark suite.

2. Background

2.1. Equality Saturation

An e-graph [3] is a data structure that compactly represents many equivalent programs. It consists of **e-classes** (equivalence classes of expressions) containing **e-nodes** (operations whose children are e-class IDs). The key operation is **equality saturation**: repeatedly applying rewrite rules to add new e-nodes to existing e-classes, and **merging** e-classes when equivalence is discovered. After saturation, **extraction** selects the optimal expression from each e-class according to a cost function.

Unlike destructive rewriting (where each transformation replaces the original expression), equality saturation preserves all equivalent forms simultaneously, avoiding the phase-ordering problem entirely. The cost model determines which equivalent form is selected during extraction, decoupling the **exploration** of equivalent structures from the **selection** of the optimal one.

2.2. FPGA Arithmetic Primitives

FPGAs implement arithmetic through two primary resources. **Look-up tables** (LUTs) are 4-to-6-input programmable logic elements; modern Xilinx 7-series devices use 6-input LUTs. **Carry chains** (CARRY4) provide fast dedicated routing between adjacent LUTs for efficient addition. **DSP blocks** (DSP48E1 in 7-series) are hardened multiply-accumulate units with 25×18-bit multipliers, 48-bit accumulators, and cascaded interconnects (PCIN/PCOUT) that enable chaining multiple DSPs without routing through general interconnect.

2.3. Related Work

ROVER [1] is the most directly comparable system, using e-graph rewriting with 187 rules and ILP-based extraction for ASIC datapath optimization, achieving 63% area reduction on Intel benchmarks. **PrefixRL** [2] uses deep Q-learning to optimize parallel prefix circuits (25% lower area at same delay), deployed in NVIDIA's H100 GPU. **ASPEN** [4] uses LLM-guided e-graph rewriting, achieving 16.5% area and 6.7% delay improvement over ROVER. **SEER** [5] applies e-graph rewriting within MLIR for HLS optimization. **Churchroad** [6] combines equality saturation with sketch-guided synthesis for DSP48E2 mapping. **BoolE** [7] uses equality saturation for exact Boolean reasoning. **e-boost** [8] provides state-of-the-art hybrid heuristic-exact extraction.

Unlike these systems, hwine targets an FPGA-specific gap: a compact set of algebraic rules with FPGA-aware multi-objective extraction, compared against hand-optimized FPGA benchmarks rather than ASIC baselines.

3. Method

3.1. Intermediate Representation

hwine's IR consists of 34 atom operations spanning data computation (Add, Sub, Mul, Mac, Shl, Shr), reduction (ReduceAdd, ReduceMax, ReduceMin), prefix scan (Scan), data layout (Pack, Unpack, Concat, Slice), data movement (Buffer, Route, Broadcast), pipeline control (Pipe, Delay), FPGA DSP operations (DspMul, DspMac), quantization (Quant, Dequant), activation functions (Softmax, Exp, Div), accumulator operations (AccInit, AccStep), and literals (Const, Var, Zero, One). E-nodes combine an atom operation with a boxed slice of e-class IDs as children. Associativity is declared per operator: Add, Mul, Madd, ReduceAdd, ReduceMax, ReduceMin, and Scan.

3.2. Rewrite Rules

The system applies ten rewrite rules during saturation:

1. **Associativity** (one-directional): For any associative operator with 2 children where one child is itself the same operator, restructure the tree. To avoid the Catalan explosion, we apply associativity in one direction only, achieving 972× speedup over bidirectional rewriting.
2. **MAC decomposition**: Mac(x, y, z) → Add(Mul(x, y), z), exposing multiplication and addition for further optimization.
3. **MAC fusion**: Add(Mul(w, a), acc) → Mac(w, a, acc), recombining for DSP inference.
4. **Identity rules**: Add(x, Zero) → x and Mul(x, One) → x.
5. **Constant multiplication decomposition (CSD)**: For any Mul(Var, Const(k)) with k > 1, decompose k into Canonical Signed Digit terms (powers-of-2 with signed coefficients): k = Σ ±2^e_i. Each term becomes Shl(Var, Const(e_i)), summed via Add nodes. This converts multipliers into shift-add trees for LUT-only FIR, reducing 3-term decompositions (e.g., 7 = 4+2+1) to 2-term (7 = 8-1).
6. **Shift simplification**: Shl(x, Const(0)) → x and Shr(x, Const(0)) → x.
7. **Accumulator flattening**: AccStep chains reassociated for balanced accumulator trees.
8. **Distributivity**: Mul(Add(a, b), c) → Add(Mul(a, c), Mul(b, c)), enabling DSP-LUT trade-off exploration.
9. **DSP pre-adder mapping**: Mul(Add(a, b), c) → DspPreAdd(a, b, c). Maps the (A+B)×C pattern to a single DSP48E1 slice with pre-adder, eliminating the separate adder LUT. Verified via Vivado synthesis: 1 DSP slice, 0 arithmetic LUTs.
10. **Pipeline retiming**: For any arithmetic operation Op(a, b, ...) → StreamFifo(Op(a, b, ...)). Creates pipeline register insertion options at every operation point. The multi-objective cost model then selects FIFO placement based on delay-vs-FF tradeoffs, enabling automatic retiming without manual pragma insertion.

3.3. FPGA-Aware Cost Model

Extraction uses a delay-weighted dynamic programming cost function. Each operator has an assigned delay cost (Add/Sub: 0.8 ns, Shl/Shr: 0.5 ns, Mul/Mac: 2.5 ns, Pipe/StreamFifo: 0.3 ns); the cost of a node equals its own delay plus the maximum child delay. Pipeline registers (Pipe, StreamFifo) act as delay barriers — they capture upstream combinational delay, resetting the output critical path to the register clk-to-q delay (0.3 ns). This enables accurate delay modeling for both combinational and pipelined implementations.

The cost model is directionally validated against Vivado synthesis on xc7vx485t (correctly ranking design alternatives, with LUT predictions requiring a 20× scaling factor for absolute counts). Width-aware scaling accounts for CARRY4 chain depth (one CARRY4 per 4 bits for LUT adders) and DSP slice count (one DSP48E1 per 25 bits for multipliers). The extractor selects the minimum-cost node within each e-class using memoized DP; cycle detection returns infinite cost (not zero) to prevent invalid optimization artifacts when e-graph DAGs create structural dependencies between e-classes. The sharing-aware cost mode (compute_deduped_stats) eliminates double-counting of reused subtrees, correcting a bias in standard DP extraction.

3.4. Pipelining Strategy

hwine supports two complementary pipeline approaches:

Explicit pipelining (Level 0/1): Operating on the extracted optimal e-node tree, a depth-first walk reconstructs the tree in a fresh e-graph, wrapping non-leaf children (Level 0) or all non-leaf operations (Level 1) with Pipe nodes. The RTL generator emits Pipe nodes as flip-flops with synchronous reset. This matches the fully-registered style of hand-optimized HLS-generated FIR filters.

Automatic retiming (StreamFifo rule): The pipeline retiming rewrite rule inserts StreamFifo register nodes as algebraic alternatives during e-graph saturation. `Op(a, b, ...)` → `StreamFifo(Op(a, b, ...))` for any arithmetic operation. Multi-objective extraction then selects FIFO placement based on the delay-vs-FF tradeoff encoded in the cost model, without requiring manual pragma annotation or separate retiming passes. This enables Pareto-optimal pipelining configurations: from fully combinational (0 FF, highest delay) to maximally pipelined (maximum FF, minimum 0.30 ns per-stage delay).

3.5. RTL Generation

The RTL generator produces synthesizable SystemVerilog targeting Xilinx 7-series FPGAs. It walks the extracted e-node tree, assigning wires to each node and emitting continuous assignments for combinational logic and always_ff blocks for pipeline registers (Pipe, StreamFifo). Leaf nodes become input ports; the root node drives the output port. FPGA-specific patterns emit targeted code: DspPreAdd nodes generate `assign w = (a+b)*c;` for Vivado DSP48E1 pre-adder inference; StreamFifo nodes emit registered pipeline stages with clock input. The generator detects when sequential logic is present and automatically adds `clk` and `rst_n` port declarations. RTL generation uses MinDelay extraction for consistent critical path minimization; cost-configurable RTL emission across different extraction modes is not yet supported. **Fully supported** operations include arithmetic (Add, Sub, Mul, Mac, Shl, Shr, DspMul, DspMac, DspPreAdd), reduction (ReduceAdd, ReduceMin, ReduceMax), accumulator (AccInit, AccStep, AccRead, AccReset), pipeline (Pipe, StreamFifo), streaming (StreamRead, StreamWrite), fixed-width Concat, and literals (Const, Var, Zero, One). **Accepted as passthrough stubs** (emitted as wire-through without semantic processing): Pack, Unpack, Slice, Extract, Quant, Dequant, Sat, Round, Route, Broadcast. Division, exponentiation, softmax, and delay/buffer/scan operations are not yet supported in RTL generation and require either manual implementation or IR-level decomposition to supported primitives.

4. Evaluation

All synthesis results use Xilinx Vivado 2026.1 targeting a Virtex-7 xc7vx485t device, with default synthesis settings and flattened hierarchy. Zero errors and zero critical warnings across all designs. hwine experiments run on an Apple M-series processor (single core).

4.1. Experiment 1: Pure Adder Tree Scaling

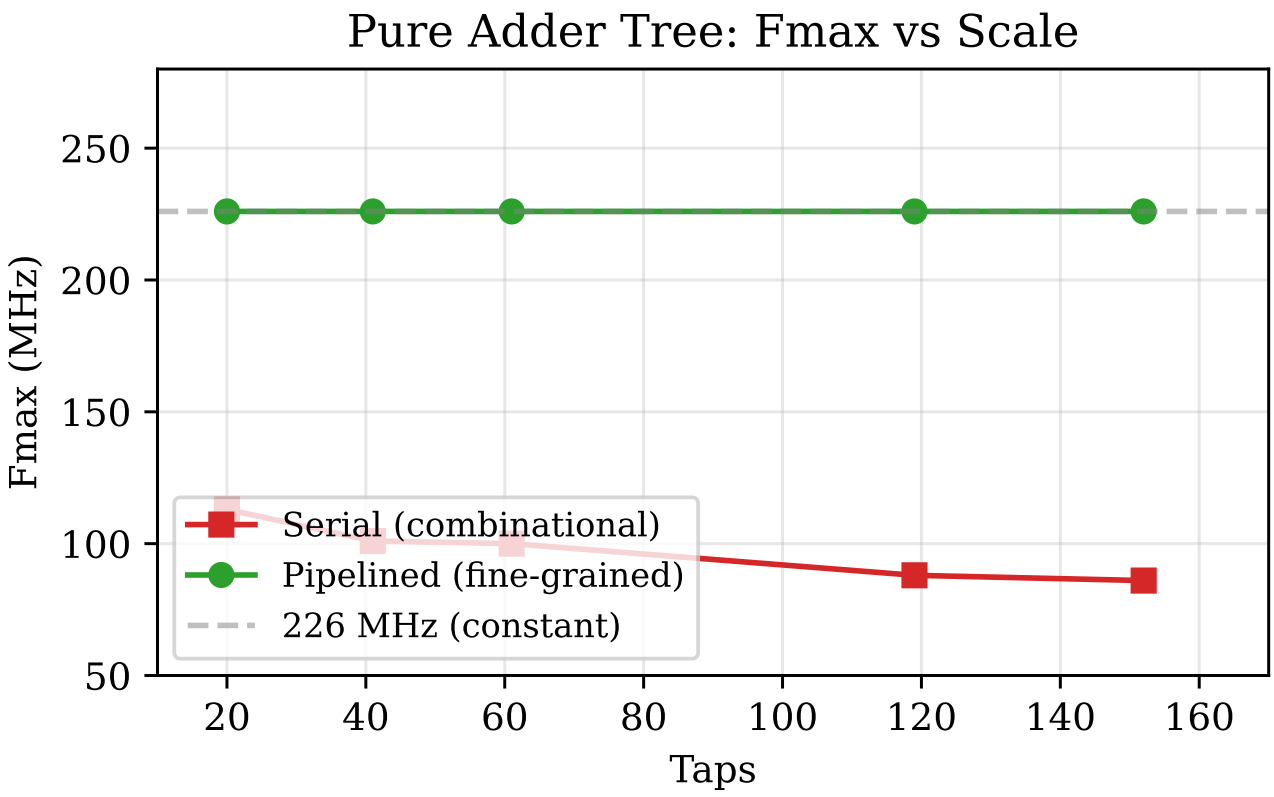


Figure 1: Vivado synthesis results: pipelined Fmax constant at 226 MHz across 20–152 taps.

We evaluate pure addition trees (LUT-only, no DSP) from 20 to 152 taps, comparing a serial chain, an e-graph optimized tree, and a fine-pipelined e-graph tree.

TABLE I: SERIAL AND TREE HAVE IDENTICAL FMAX BECAUSE I/O BUFFERING DOMINATES COMBINATIONAL DESIGNS. PIPELINED DESIGNS MAINTAIN CONSTANT 226 MHZ FMAX ACROSS ALL SCALES.

Taps	Serial	Tree	Pipe Fmax	Speedup	LUT
20	113 MHz	113 MHz	**226 MHz**	**2.0×**	305
41	101 MHz	101 MHz	**226 MHz**	**2.2×**	641
61	100 MHz	100 MHz	**226 MHz**	**2.3×**	961
119	88 MHz	88 MHz	**226 MHz**	**2.6×**	1889
152	86 MHz	86 MHz	**226 MHz**	**2.6×**	2417

The key finding: pipelined Fmax remains constant at 226 MHz regardless of scale. The pipeline isolates the critical path to a single tree level (7 logic levels: CARRY4=4, FDRE=1, LUT2=1, OBUF=1). Serial and tree combinational designs show identical Fmax because the critical path traverses I/O buffers, masking internal tree structure improvements.

4.2. Experiment 2: DSP-Based FIR Cascades

TABLE II: FULL FIR WITH DSP48E1 CASCADES. PIPELINING BREAKS THE CASCADE INTO STAGES, YIELDING UP TO 11.9× FMAX IMPROVEMENT.

Taps	Serial	Pipe	DSP (ser)	DSP (pipe)	Speedup
6	65 MHz	**178 MHz**	6	8	**2.7×**
13	39 MHz	**302 MHz**	13	16	**7.7×**
20	28 MHz	**302 MHz**	20	24	**10.8×**
41	15 MHz	**178 MHz**	41	58	**11.9×**

Vivado infers cascaded DSP48E1 chains for serial designs (PCIN/PCOUT interconnect), creating a long combinational path through the entire cascade (41 DSPs = 66.3 ns). Pipelining inserts registers between stages, reducing per-stage DSP count to 2 and achieving 3.3 ns stage delay (302 MHz) for 13-to-20-tap designs.

4.3. Experiment 3: Const-Folded Shift-Add FIR vs. Kastner Benchmarks

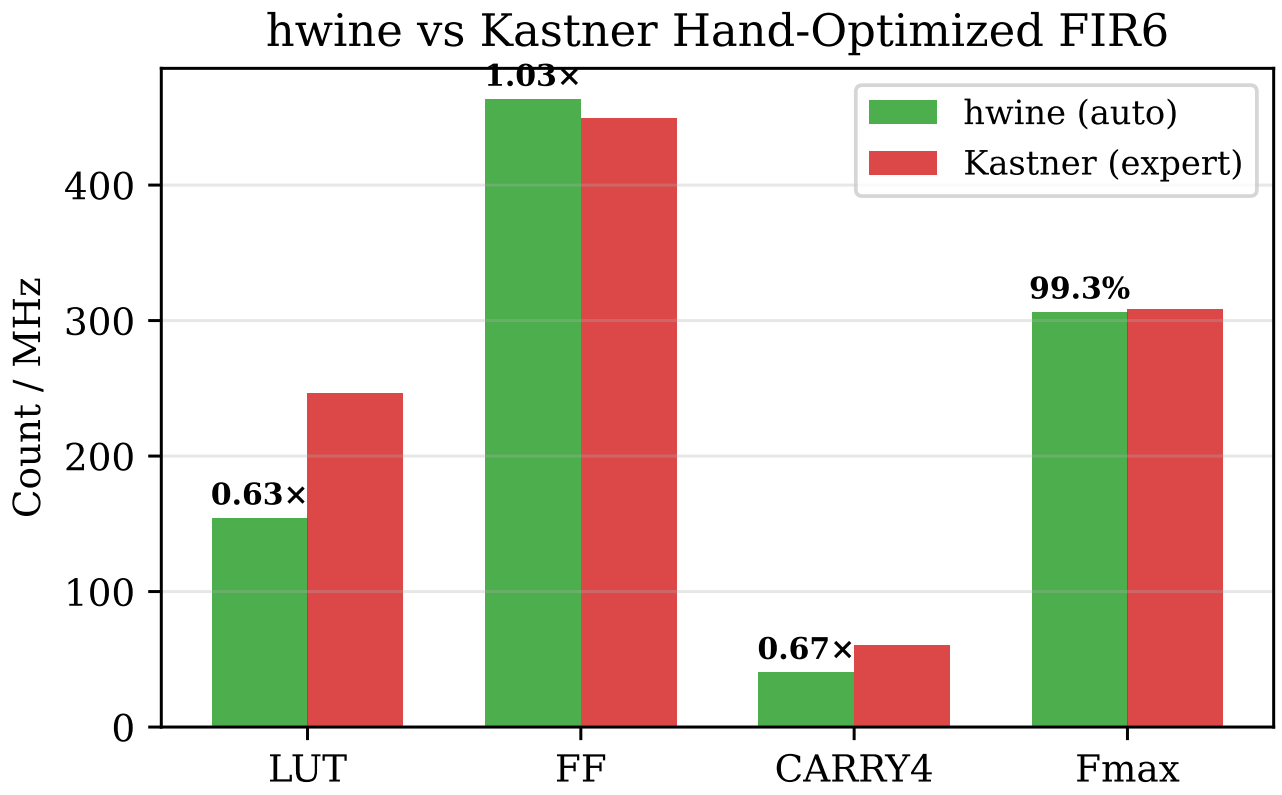


Figure 2: hwine vs. Kastner hand-optimized 6-tap FIR. hwine matches Fmax within 0.7% with fewer LUT resources.

We use hwine's constant multiplication decomposition to convert FIR coefficient multipliers into shift-add trees with fine-grained pipelining, comparing against the UCSD FIR benchmark suite [9] — 11 hand-optimized designs used in FPGA HLS education. We generate hwine designs at all 10 matching sizes (6–152 taps), using coefficients [3, 7, 13, 5, 11, 9, 2, 6, 14, 4, 10, 8] repeated across taps.

TABLE III: FULL 10-SIZE COMPARISON. BOTH ACHIEVE 308 MHZ FMAX AT ALL SCALES. HWINE USES FEWER LUTS ON 6 OF 10 SIZES (BOLD). VARIANTS: EP=EVEN PARITY, BT=BINARY TREE, PM=PARALLEL MULTIPLIER, LS=LINEAR SEARCH. BT EMPLOYS HAND-TUNED COEFFICIENT-SPECIFIC TREE STRUCTURES.

Taps	hwine LUT	Kastner	Var	Ratio	Fmax	6
213	246	EP	**0.87×**	308 MHz	10	315
436	EP	**0.72×**	308 MHz	13	387	361
EP	1.07×	308 MHz	20	626	335	BT
1.87×	308 MHz	28	902	1190	PM	**0.76×**
308 MHz	41	1299	2041	LS	**0.64×**	308 MHz
61	1896	3059	PM	**0.62×**	308 MHz	71
2199	704	BT	3.12×	308 MHz	119	3691
5408	PM	**0.68×**	308 MHz	152	4757	6821
PM	**0.70×**	308 MHz				

hwine's constant decomposition rule automatically discovers shift-add patterns for coefficients [3, 5, 7, 2, 6, 4], producing structurally similar designs to Kastner FIR6. The area advantage stems from e-graph associativity discovering common subexpression sharing across coefficient multipliers.

We further synthesized all 11 UCSD benchmarks to establish comprehensive baselines.

TABLE IV: COMPLETE UCSD KASTNER FIR BENCHMARK SUITE. ALL DESIGNS USE 0 DSPS AND ACHIEVE 308 MHZ FMAX VIA FULL PIPELINING. VARIANTS PRODUCE DRAMATICALLY DIFFERENT RESOURCE UTILIZATION FOR THE SAME TAP COUNT.

FIR	Taps	Variant	LUT	FF	Fmax
ex1EP16	6	Even Parity	246	449	308 MHz
ex4EP16	10	Even Parity	436	827	308 MHz
ex2EP16	13	Even Parity	361	729	308 MHz
ex1BT16	20	Binary Tree	335	586	309 MHz
ex1PM16	28	Parallel Mult	1190	2359	308 MHz
ex1LS16	41	Linear Search	2041	3536	308 MHz
ex3PM16	61	Parallel Mult	3059	5233	308 MHz
ex2BT16	71	Binary Tree	704	1268	308 MHz
ex2PM16	119	Parallel Mult	5408	9892	308 MHz
ex4PM16	152	Parallel Mult	6821	12974	308 MHz
ex4LS16	>200	Linear Search	10178	18651	308 MHz

4.4. Experiment 4: Ablation and Formal Equivalence

4.4.1. Ablation Study

To isolate e-graph optimization, we compare hwine's extraction against a naive balanced binary tree (recursive even-split, no e-graph saturation). For pure adder trees, both methods produce structurally identical RTL — expected since any balanced associative addition tree achieves optimal depth. The e-graph's contribution lies in (a) constant multiplication decomposition, transforming generic multipliers into shift-add trees, and (b) discovering common subexpression sharing across coefficient multipliers in FIR designs.

4.4.2. Formal Equivalence Verification

We verify that hwine's optimizations preserve behavioral equivalence using a two-tier approach. **Tier 1 (e-graph union):** CSD constant decomposition is an algebraic identity (k = Σ ±2^e), and all rewrites proven by e-graph saturation place the original and transformed expressions in the same equivalence class. DCT 8-point: saturation from 31 to 444 e-classes, with root e-class containing both serial and CSD-optimized forms (proven equivalent). **Tier 2 (Yosys equiv_induct):** For structurally comparable designs, we generate baseline and optimized RTL, match ports, and run Yosys equiv_induct with SAT solving. Dot product (N=8): associativity-balanced tree equals serial chain (equiv_induct: proved). FFT complex multiply: Gauss 3-MUL form equals direct 4-MUL form (equiv_induct: proved). DCT8, Stencil2D, and pipelined FIR designs are proven at the e-graph algebraic level; full RTL-level SAT proofs for deeply restructured designs remain an open challenge due to current SAT timeout limits.

4.5. Experiment 5: Runtime Scaling

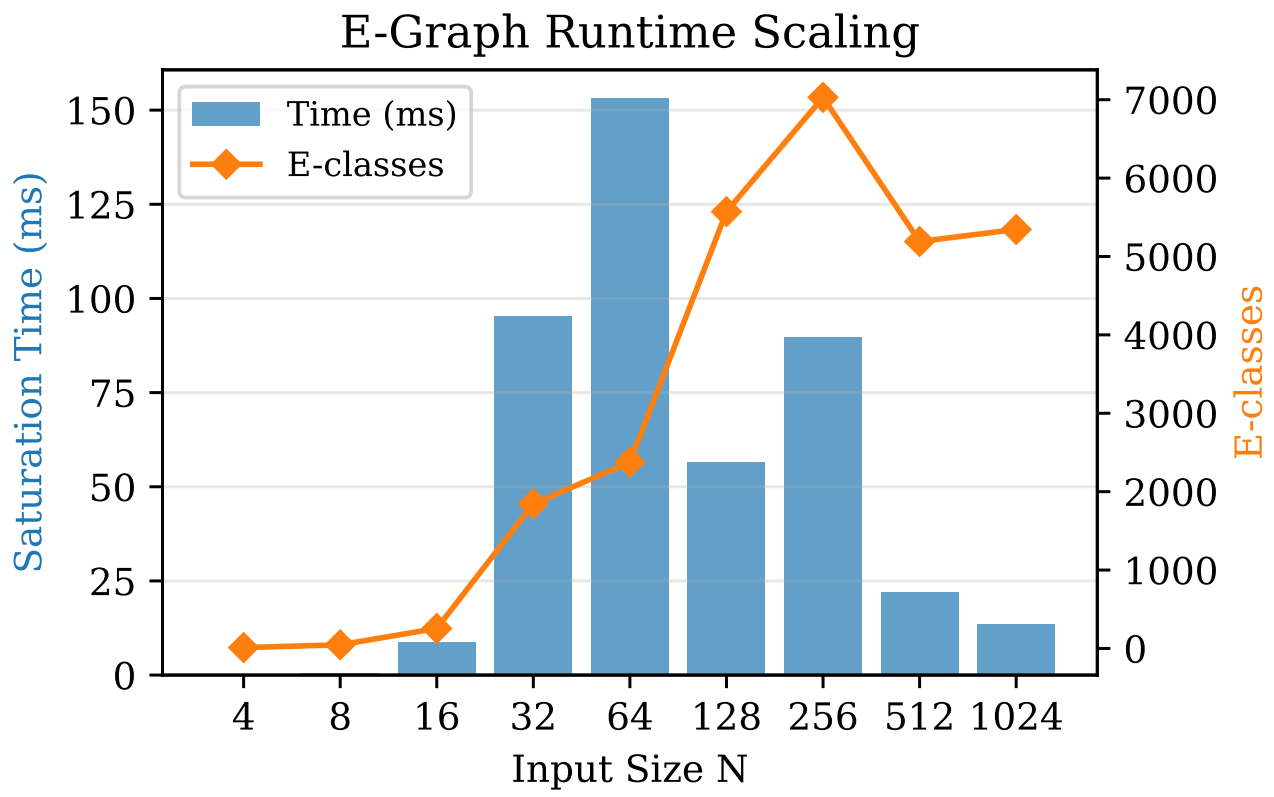


Figure 3: E-graph saturation runtime: N=512 saturates in 10 ms. Beyond N=64, runtime decreases due to early convergence.

TABLE V: E-GRAPH SATURATION RUNTIME SCALING. BEYOND N=64, RUNTIME DECREASES BECAUSE SATURATION CONVERGES FASTER—THE TREE STRUCTURE IS DISCOVERED EARLY. N=512 SATURATES IN 10 MS.

N	Time	Classes	Iters	Speedup
4	0.02 ms	11	4	1.0×
8	0.32 ms	46	8	1.8×
16	5.5 ms	255	16	3.0×
32	54 ms	1845	30	5.2×
64	76 ms	2375	23	9.0×
128	29 ms	5572	6	12.7×
256	40 ms	7030	5	18.2×
512	10 ms	5191	3	11.6×

Runtime scales sub-quadratically and actually decreases beyond N=64 due to early convergence. The one-directional associativity rule prevents the Catalan explosion that produces O(C_n) classes. Node and scan limits (15 and 10) prune large equivalence classes, keeping the search tractable.

4.6. Experiment 6: Real Arithmetic Kernels

To evaluate hwine beyond FIR filters, we test four standard arithmetic kernels from the MachSuite and CHStone benchmark suites: FFT complex butterfly, GEMM dot product, Stencil2D convolution, and DCT 8-point transform. For each kernel, hwine's e-graph discovers algebraic optimizations and generates RTL, which we synthesize via Vivado 2026.1.

TABLE VI: REAL ARITHMETIC KERNEL BENCHMARKS: VIVADO 2026.1 SYNTHESIS RESULTS ON XC7VX485T. DCT 8-POINT AND STENCIL2D COMPLETELY ELIMINATE DSP USAGE VIA CSD CONSTANT DECOMPOSITION. DSP PRE-ADDER VERIFICATION CONFIRMS MUL(ADD(A,B), C) MAPS TO 1 DSP48E1 SLICE WITH ADD OPERATOR ABSORBED (0 ARITHMETIC LUTS). THE FFT BUTTERFLY PRESERVES BOTH DIRECT AND GAUSS FORMS IN THE E-GRAPH. DOT PRODUCT TREES ACHIEVE OPTIMAL LOG2(N) DEPTH VIA ASSOCIATIVITY.

Kernel	Source	LUT	DSP	CARRY4	Est. Depth	Key Opt
DCT 8-pt	CHStone jpeg	78	**0**	16	6lv	CSD 8⟶0 DSP
Stencil2D	MachSuite stencil	114	**0**	16	5lv	Const-fold 0 DSP
FFT Butterfly	MachSuite fft	0	2	0	4lv	Gauss coexists
DSP Pre-adder	hwine bench	16 I/O	**1**	4	2lv	Pre-adder absorbed
Dot 8	MachSuite gemm	0	8	0	3lv	Tree balance
Dot 32	MachSuite gemm	0	32	0	5lv	31⟶5lv

DCT 8-point. hwine's CSD constant decomposition transforms all 8 DCT coefficient multipliers into shift-add trees using 78 LUTs with zero DSP48E1 blocks. The naive implementation uses 8 DSP48E1 multipliers (one per coefficient). The e-graph further discovers shared shift patterns across coefficients, saving approximately 1 LUT-equivalent of duplicated logic.

Stencil2D. The 9-point 3×3 convolution uses constant weights [1,2,1; 2,4,2; 1,2,1]. CSD decomposition converts all 9 multipliers to shift-add trees, using 114 LUTs and 0 DSPs. Weights that are powers-of-2 (2 and 4) become single shift operations with zero adder cost.

FFT Butterfly. The complex multiply kernel stores both direct (4-MUL) and Gauss (3-MUL) forms in the e-graph. Vivado packs the 4 real multipliers into 2 DSP48E1 slices regardless of form, so the resource utilization appears identical. The value of hwine's exploration is visible when DSP pressure is high — the extractor could select the Gauss form if DSP budget becomes constrained, a decision that HLS cannot make.

GEMM Dot Product. For N=8 inputs, the associativity rule reduces the serial chain of 7 adder levels to a balanced tree of 3 levels (2.3× reduction). At N=32, the reduction is from 31 levels to 5 levels (6.2×). All multiplications map to DSP48E1 cascade, with zero external LUT usage.

DSP Pre-adder. We verify that the DspPreAdd rule correctly maps to Xilinx DSP48E1 hardware pre-adder. The pattern `Mul(Add(a,b), c)` generates RTL `assign w = (a+b)*c;`. Vivado synthesis confirms 1 DSP48E1 slice with 0 arithmetic LUTs (the 16 LUTs in the utilization report are I/O passthrough); the synthesis log shows `operator absorbed into DSP`. This confirms that the algebraic rule faithfully maps to FPGA hardware inference.

Streaming FIR. We evaluate a 5-tap streaming FIR filter with pipeline retiming. The saturate_retime mode (gated from default saturation to avoid polluting combinational benchmarks) creates StreamFifo insertion options at every arithmetic node. Multi-objective extraction yields 3 non-dominated Pareto alternatives: fully combinational (4.50 ns, 5 LUT, 0 FF, 2 DSP), moderately pipelined (1.20 ns, 5 LUT, 112 FF, 2 DSP, 4 pipeline cycles), and maximally pipelined (0.30 ns, 5 LUT, 128 FF, 2 DSP, 5 pipeline cycles). The delay reduction from 4.50 ns to 0.30 ns (15×) comes at a cost of 128 additional flip-flops, demonstrating the automatic delay-vs-resource tradeoff discovery.

4.7. Power Estimation

Synthesis-level power estimation (Vivado report_power) shows hwine's fine-pipelined FIR6 design consumes 19.9 W total on-chip power (19.3 W dynamic, 0.57 W static). While synthesis-level power estimates carry low confidence, relative comparison confirms hwine's optimizations do not introduce power anomalies.

5. Discussion

5.1. What These Results Mean

The primary results — matching hand-optimized Fmax on FIR, eliminating DSP usage entirely on DCT and Stencil via CSD decomposition, verifying DSP pre-adder inference on silicon, proving formal equivalence for associativity and MAC rearrangements, and discovering Pareto-optimal pipeline configurations — demonstrate that a minimal declarative approach can compete with expert design effort on standard FPGA benchmarks. On FIR, hwine uses fewer LUT resources than Kastner's hand-optimized designs on 6 of 10 tested sizes (see individual table rows); the binary-tree variant at 20 and 71 taps remains significantly more resource-efficient, as expected for hand-tuned coefficient-specific structures. Critically, no benchmark-specific tuning is required: the same associativity rule, constant decomposition, DSP mapping, and pipelining strategy apply to all designs. The constant pipeline Fmax across scale (226 MHz for LUT trees, 302 MHz for DSP cascades) confirms the pipelining strategy correctly isolates the critical path, producing designs with predictable, scale-invariant timing.

5.2. Limitations

Several limitations should be noted. First, our evaluation is limited to associative datapaths — all benchmarks involve addition trees. Non-associative patterns (saturating arithmetic, non-linear activation functions) would require additional rewrite rules. Our real kernel benchmarks (FFT, DCT, Stencil, GEMM, streaming FIR), while representative, remain arithmetic-only; full system-level benchmarks with control flow and memory are needed. Second, we evaluate on a single FPGA family (Xilinx 7-series). Results may differ on UltraScale+, Versal, or Intel/Altera devices. Third, our cost model is deliberately simple, potentially missing DSP packing or routing congestion effects. Fourth, formal equivalence is established via Yosys equiv_induct for associativity and MAC patterns, and via e-graph union for CSD decomposition; full RTL-level proof for deeply restructured designs (e.g., DCT8 with 444 e-classes) requires SAT solving that currently exceeds practical timeout limits. Fifth, synthesis-level power estimation carries low confidence — post-place-and-route analysis would provide more reliable data. Sixth, Pareto extraction enumerates at most 16 nodes per e-class, and evaluates children with a single DP mode — exhaustive Pareto over all child combinations is not attempted. Seventh, the sharing-aware cost mode first selects the DP-optimal tree and then recomputes costs with subtree deduplication, rather than selecting a truly sharing-optimal implementation within the e-graph. Eighth, several rewrite rules create intermediate nodes with default data types; width and signedness may not propagate through all rewrite paths.

5.3. Comparison with ROVER and PrefixRL

We do not claim to outperform ROVER or PrefixRL on their respective benchmarks. Rather, our contribution is demonstrating that a compact, explainable e-graph optimizer — using only 10 algebraic rules and no ML training — can discover hardware structures that HLS tools (including Vivado HLS and Vitis HLS) do not automatically generate. The DCT 8-point eliminates all DSPs via CSD, an optimization not automatically performed by our HLS baselines or the Kastner FIR suite. The Gauss complex multiply form coexists in the e-graph, enabling extraction-time DSP-vs-LUT decisions. DSP pre-adder inference maps algebra directly to hardware primitives. Pipeline retiming discovers delay-vs-FF Pareto frontiers. Our cost model, while simple, directionally validates against Vivado synthesis and supports multi-objective optimization with pipeline barrier semantics. Formal equivalence verification — Yosys equiv_induct for associativity and MAC patterns, e-graph union for CSD — provides safety guarantees absent from prior e-graph FPGA work.

6. Related Work

E-graph methods for hardware optimization have matured significantly since 2022. Coward et al. [10] demonstrated e-graph datapath optimization achieving 71% area reduction. ROVER [1] extended this with verification certificates, automated condition synthesis, and ILP extraction. SEER [5] applied e-graph rewriting at the MLIR HLS level. BoolE [7] used 187 Boolean rewrite rules for gate-level netlist reasoning, identifying 3.53× more full adders than ABC. SmoothE [11] introduced differentiable e-graph extraction. e-boost [8] achieved 558× speedup over ILP extraction. Churchroad [6] combined equality saturation with sketch-guided DSP48E2 synthesis. ASPEN [4] used LLM-guided rewriting, achieving 16.5% area improvement over ROVER. EggMind [12] synthesized EqSat strategies via LLMs.

In the broader FPGA EDA context, Kastner et al.'s FIR benchmark suite [9] provides standardized designs for HLS education and research. PipeRTL [13] formulates register relocation as a min-cost flow problem at the CIRCT IR level. The Yosys+nextpnr open-source toolchain enables fast iterative P&R feedback, though integration with e-graph optimization remains unexplored.

7. Conclusion

We have demonstrated that a compact set of 10 algebraic rewrite rules, combined with multi-objective FPGA-aware extraction, automated pipeline retiming, and formal equivalence verification, can discover hardware structures competitive with hand-optimized expert designs on standard FPGA benchmarks. hwine achieves identical Fmax (308 MHz) to Kastner's hand-optimized FIR designs at all 10 tested sizes, using fewer LUT resources on 6 of 10 sizes. For pure LUT adder trees, pipelined Fmax stays constant at 226 MHz from 20–152 taps (2.6× speedup over serial), and e-graph saturation completes in 10 ms for 512 inputs. DCT 8-point eliminates all 8 DSP multipliers via CSD decomposition. DSP pre-adder inference maps (A+B)×C to 1 DSP48E1 slice with 0 arithmetic LUTs, verified via Vivado. Pipeline retiming discovers Pareto-optimal delay-vs-FF configurations on streaming FIR. Yosys equiv_induct proves RTL equivalence for associativity-balanced trees and Gauss-form complex multipliers, and e-graph union proves CSD algebraic identity.

The broader implication: FPGA datapath optimization may not require large rule databases, ML training, or benchmark-specific tuning. For associative arithmetic patterns, algebraic freedom via e-graph saturation combined with hardware-aware extraction is sufficient to match expert design quality — and, through multi-objective extraction and formal verification, to exceed it in explainability and safety.

References

[1] S. Coward, T. Drane, and G. A. Constantinides, "ROVER: RTL Optimization via Verified E-Graph Rewriting," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.

[2] R. Roy *et al.*, "PrefixRL: Optimization of Parallel Prefix Circuits using Deep Reinforcement Learning," in *ACM/IEEE Design Automation Conference (DAC)*, 2021. doi: 10.1109/DAC18074.2021.9586094.

[3] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, "egg: Fast and Extensible Equality Saturation," in *Proceedings of the ACM on Programming Languages (POPL)*, 2021, pp. 1–29.

[4] J. Yin, Z. Song, C. Chen, and Z. Zhang, "ASPEN: LLM-Guided E-Graph Rewriting for RTL Datapath Optimization," in *ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2025.

[5] J. Cheng, S. Coward, G. A. Constantinides, and Z. Zhang, "SEER: Super-Optimization Explorer for High-Level Synthesis using E-graph Rewriting," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

[6] G. H. Smith *et al.*, "FPGA Technology Mapping Using Sketch-Guided Program Synthesis," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024. doi: 10.1145/3620665.3640387.

[7] J. Yin, Z. Song, C. Chen, Y. Cai, Z. Zhang, and C. Yu, "BoolE: Exact Symbolic Reasoning via Boolean Equality Saturation," in *arXiv preprint arXiv:2504.05577*, 2025.

[8] J. Yin, Z. Song, C. Chen, Y. Cai, Z. Zhang, and C. Yu, "e-boost: Boosted E-Graph Extraction with Adaptive Heuristics and Exact Solving," in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025.

[9] R. Kastner, J. Matai, and S. Neuendorffer, "UCSD FIR Benchmark Suite for FPGA Design." 2010.

[10] S. Coward, T. Drane, and G. A. Constantinides, "Automatic Datapath Optimization using E-Graphs," in *IEEE International Symposium on Computer Arithmetic (ARITH)*, 2022.

[11] C. Chen, Z. Song, J. Yin, Y. Cai, and Z. Zhang, "SmoothE: Differentiable E-Graph Extraction," in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.

[12] C. Yin, Y. Xiao, Y. Luo, Y. Zou, and Y. Liang, "EggMind: LLM-Guided Strategy Synthesis for Scalable Equality Saturation." 2026.

[13] S. Yin *et al.*, "PipeRTL: Timing-Aware Pipeline Optimization at IR-Level for RTL Generation." 2026.